CYRANO

Complete Technical Architecture & Implementation Guide

AI-Powered Dating Assistant

Executive Summary

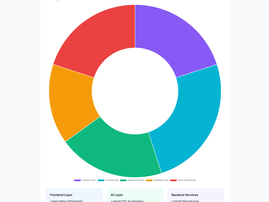
Vision

CYRANO is revolutionary AI-powered dating assistant that combines the eloquence of Cyrano de Bergerac with cutting-edge artificial intelligence. The system provides real-time conversation through expert-level coaching, personalized communication coaching, and comprehensive relationship coverage services.

Key Differentiators

- Agents-to-Agent Matching
- Real-time Conversation Coaching
- Privacy-First Architecture
- AI Relationship Coverage

System Architecture Overview



Frontend Layer
- React Native (Mobile)
- Progressive Web App
- Real-time Dashboard
- Voice/Video Integration

AI Layer
- OpenAI GPT-4o Integration
- Google Vertex AI
- Custom Conversation Models
- Vector Embeddings

Backend Services
- Supabase Microservices
- PostgreSQL + pgvector
- Redis Cache
- Google Cloud Run

CYRANO AI Agent Core

Agent Personality Architecture

```
class CyranoAgent:
    def __init__(self, user_profile, skill_level):
        self.personality = {
            'base_traits': {
                'eloquent': 0.95,
                'patient': 0.90,
                'wise': 0.85,
                'romantic': 0.80,
                'protective': 0.75
            },
            'adaptive_traits': {
                'formality': 0.6,
                'humor': 0.7,
                'boldness': 0.5,
                'vulnerability': 0.8
            }
        }

    async def find_matches(self, user_query: Conversation):
        """Main matching agent to-agent discovery"""

        # Step 1: Generate user representation embedding
        user_embed = await self._create_agent_embedding(user_agent)

        # Step 2: Find potential candidates using vector similarity
        candidates = await self.vector_db.similarity_search(user_embed, k=100)

        # Step 3: Deep compatibility analysis for each potential match
        matches = []
        for candidate in candidates:
            compatibility = await self._analyze_agent_conversation(
                user_agent, candidate_agent
            )
            if compatibility.score > 0.7:
                matches.append({
                    'agent': candidate_agent,
                    'compatibility': compatibility,
                    'conversation_preview': compatibility.preview
                })

        return sorted(matches, key=lambda x: x['compatibility'].score, reverse=True)

    def _create_agent_embedding(self, agent_profile):
        """Convert agent profile to vector representation"""
        profile_text = f"{agent_profile.personality_traits} {agent_profile.communication_style} {agent_profile.publication_preferences}"
        return self.embedding_model.encode(profile_text)

    def _analyze_agent_conversation(self, agent_a, agent_b):
        """Simulate conversation between two agents"""
        conversation = []
        for turn in range(self.max_turns):
            # Agent A speaks
            response_a = self.generate_response(agent_a, agent_b, conversation)
            conversation.append({'speaker': 'A', 'text': response_a})

            # Agent B responds
            response_b = self.generate_response(agent_b, agent_a, conversation)
            conversation.append({'speaker': 'B', 'text': response_b})

            # Analyze compatibility for this turn
            compatibility = self.analyze_compatibility(conversation)
            if compatibility.confidence > 0.95:
                break

        return {
            'conversation': conversation,
            'score': compatibility.score,
            'reasons': compatibility.reasons,
            'preview': self.generate_preview(conversation)
        }
```

Core Capabilities

Message Coaching
Generate eloquent, personalized messages that reflect user's authentic voice

Conversation Analysis
Real-time analysis of conversation patterns and compatibility

Relationship Advice
Expert guidance on relationship building and dating strategies

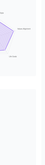
Safety Monitoring
Detection of manipulation, harm, or inappropriate communication

Agent-to-Agent Matching Algorithm

Matching Process Flow

1. Agent Profile Creation & Vector Embedding
2. Vector Space Coordinates Simulation
3. Multi-Dimensional Compatibility Analysis
4. Confidence Scoring & Match Ranking
5. User Presentation with AI Insights

Compatibility Factors



Core Algorithm Implementation

```
class AgentMatchingEngine:
    def __init__(self):
        self.vector_db_client = pinecone.Pinecone(api_key=os.environ['PINECONE_API_KEY'])
        self.embedding_model = sentence_transformers.SentenceTransformer('all-MiniLM-L6-v2')
        self.compatibility_weights = {
            'personality': 0.25,
            'relationship_goals': 0.20,
            'communication_style': 0.15,
            'publication_preferences': 0.10,
            'cultural_intelligence': 0.10,
            'off_limits': 0.20
        }

    async def find_matches(self, user_query: Conversation) -> List[MatchResult]:
        """Main matching agent to-agent discovery"""

        # Step 1: Generate user representation embedding
        user_embed = await self._create_agent_embedding(user_agent)

        # Step 2: Find potential candidates using vector similarity
        candidates = await self.vector_db.similarity_search(user_embed, k=100)

        # Step 3: Deep compatibility analysis for each potential match
        matches = []
        for candidate in candidates:
            compatibility = self._analyze_agent_conversation(
                user_agent, candidate_agent
            )
            if compatibility.score > 0.7:
                matches.append({
                    'agent': candidate_agent,
                    'compatibility': compatibility,
                    'conversation_preview': compatibility.preview
                })

        return sorted(matches, key=lambda x: x['compatibility'].score, reverse=True)

    def _create_agent_embedding(self, agent_profile):
        """Convert agent profile to vector representation"""
        profile_text = f"{agent_profile.personality_traits} {agent_profile.communication_style} {agent_profile.publication_preferences}"
        return self.embedding_model.encode(profile_text)

    def _analyze_agent_conversation(self, agent_a, agent_b):
        """Simulate conversation between two agents"""
        conversation = []
        for turn in range(self.max_turns):
            # Agent A speaks
            response_a = self.generate_response(agent_a, agent_b, conversation)
            conversation.append({'speaker': 'A', 'text': response_a})

            # Agent B responds
            response_b = self.generate_response(agent_b, agent_a, conversation)
            conversation.append({'speaker': 'B', 'text': response_b})

            # Analyze compatibility for this turn
            compatibility = self.analyze_compatibility(conversation)
            if compatibility.confidence > 0.95:
                break

        return {
            'conversation': conversation,
            'score': compatibility.score,
            'reasons': compatibility.reasons,
            'preview': self.generate_preview(conversation)
        }

    def _analyze_compatibility(self, conversation):
        """Multi-dimensional compatibility analysis"""
        analysis = {
            'personality': self.analyze_personality(conversation),
            'relationship_goals': self.analyze_relationship_goals(conversation),
            'communication_style': self.analyze_communication_style(conversation),
            'publication_preferences': self.analyze_publication_preferences(conversation),
            'cultural_intelligence': self.analyze_cultural_intelligence(conversation),
            'off_limits': self.check_off_limits(conversation)
        }

        # Calculate weighted compatibility score
        total_score = 0
        for category, weight in self.compatibility_weights.items():
            if category != 'off_limits':
                total_score += analysis[category].score * weight

        # Apply off-limits penalty
        if analysis['off_limits'].violations:
            total_score *= 0.1

        return CompatibilityResult(
            score=total_score,
            breakdown=analysis,
            recommendation=self.generate_recommendation(analysis)
        )
```

Database Architecture

Enhanced Data Models

```
from sqlalchemy import Column, String, Float, JSON, Text, ForeignKey
from sqlalchemy.dialects.postgresql import ARRAY, UUID
from sqlalchemy.orm.declarative_base import declarative_base
from pgvector.sqlalchemy import Vector

Base = declarative_base()

class User(Base):
    __tablename__ = 'users'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    email = Column(String(255), unique=True, nullable=False)
    name = Column(String(100), nullable=False)
    profile = Column(JSONB)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, onupdate=datetime.utcnow)

    # Relationships
    profile_vectors = relationship("ProfileVector", back_populates="user")
    sent_matches = relationship("Match", foreign_keys="Match.user_a_id")
    received_matches = relationship("Match", foreign_keys="Match.user_b_id")
    conversations = relationship("Conversation", back_populates="user")

class ProfileVector(Base):
    __tablename__ = 'profile_vectors'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id = Column(UUID(as_uuid=True), ForeignKey('users.id'))
    embedding = Column(Vector(384))  # sentence-transformers dimension
    personality_vector = Column(JSONB)
    relationship_vector = Column(JSONB)
    communication_vector = Column(JSONB)
    publication_vector = Column(JSONB)
    cultural_vector = Column(JSONB)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Relationships
    user = relationship("User", back_populates="profile_vectors")

class AgentProfile(Base):
    __tablename__ = 'agent_profiles'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id = Column(UUID(as_uuid=True), ForeignKey('users.id'))
    personality_traits = Column(JSONB)
    relationship_goals = Column(JSONB)
    communication_style = Column(JSONB)
    publication_preferences = Column(JSONB)
    cultural_intelligence = Column(JSONB)
    off_limits = Column(JSONB)
    agent_embedding = Column(Vector(384))
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, onupdate=datetime.utcnow)

class Match(Base):
    __tablename__ = 'matches'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_a_id = Column(UUID(as_uuid=True), ForeignKey('users.id'))
    user_b_id = Column(UUID(as_uuid=True), ForeignKey('users.id'))
    compatibility_score = Column(Float)
    conversation_preview = Column(JSONB)
    compatibility_breakdown = Column(JSONB)
    status = Column(Enum('pending', 'accepted', 'rejected', 'expired'))
    created_at = Column(DateTime, default=datetime.utcnow)
    expires_at = Column(DateTime)

    # Relationships
    user_a = relationship("User", foreign_keys="Match.user_a_id")
    user_b = relationship("User", foreign_keys="Match.user_b_id")

class Conversation(Base):
    __tablename__ = 'conversations'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id = Column(UUID(as_uuid=True), ForeignKey('users.id'))
    partner_id = Column(UUID(as_uuid=True))
    messages = Column(JSONB)
    analysis = Column(JSONB)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, onupdate=datetime.utcnow)

class UserPreferences(Base):
    __tablename__ = 'user_preferences'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id = Column(UUID(as_uuid=True), ForeignKey('users.id'))
    publication_preferences = Column(JSONB)
    relationship_goals = Column(JSONB)
    communication_style = Column(JSONB)
    off_limits = Column(JSONB)
    notification_preferences = Column(JSONB)
    privacy_settings = Column(JSONB)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, onupdate=datetime.utcnow)
```

Vector Search Implementation

```
class VectorSearchEngine:
    def __init__(self):
        self.pinecone = pinecone.Pinecone(
            api_key=os.environ['PINECONE_API_KEY']
        )
        self.index = self.pinecone.Index('cyrano-profiles')

    async def store_profile_vector(self, user_id, profile_data, embedding):
        """Store user profile vector in Pinecone"""
        try:
            # Prepare vector with metadata
            vector = {
                'id': f'{user_id}',
                'values': embedding.tolist(),
                'metadata': {
                    'user_id': user_id,
                    'personality_traits': profile_data.get('personality_traits', []),
                    'publication_preferences': profile_data.get('publication_preferences', {}),
                    'relationship_goals': profile_data.get('relationship_goals', []),
                    'cultural_intelligence': profile_data.get('cultural_intelligence', {}),
                    'created_at': datetime.utcnow().isoformat()
                }
            }

            # Upsert vector
            self.index.upsert([vector])
            logger.info(f"Successfully stored vector for user {user_id}")

        except Exception as e:
            logger.error(f"Failed to store vector for user {user_id}: {str(e)}")
            raise

    def store_profile_vector(self, profile, embedding):
        """Store profile vector in Pinecone"""
        try:
            # Prepare vector with metadata
            vector = {
                'id': f'{profile.user_id}',
                'values': embedding.tolist(),
                'metadata': {
                    'user_id': profile.user_id,
                    'personality_traits': profile.personality_traits,
                    'relationship_goals': profile.relationship_goals,
                    'communication_style': profile.communication_style,
                    'publication_preferences': profile.publication_preferences,
                    'cultural_intelligence': profile.cultural_intelligence,
                    'off_limits': profile.off_limits,
                    'created_at': datetime.utcnow().isoformat()
                }
            }

            # Upsert vector
            self.index.upsert([vector])
            logger.info(f"Successfully stored vector for user {profile.user_id}")

        except Exception as e:
            logger.error(f"Failed to store vector for user {profile.user_id}: {str(e)}")
            raise

    def find_similar_profiles(self, query_embedding, filters=None, top_k=100):
        """Find similar profiles using vector similarity"""
        try:
            # Prepare query
            query = {
                'vector': query_embedding.tolist(),
                'top_k': top_k,
                'include_metadata': True
                'include_values': False
            }

            # Add filters if provided
            if filters:
                query['filter'] = filters

            # Execute query
            results = self.index.query(**query)

            # Extract matches with scores
            matches = []
            for match in results['matches']:
                matches.append({
                    'id': match['id'],
                    'score': match['score'],
                    'metadata': match['metadata']
                })

            return matches

        except Exception as e:
            logger.error(f"Failed to query similar profiles: {str(e)}")
            raise

    def delete_profile_vector(self, user_id):
        """Delete user profile vector"""
        try:
            self.index.delete([f'{user_id}'])
            logger.info(f"Successfully deleted vector for user {user_id}")

        except Exception as e:
            logger.error(f"Failed to delete vector for user {user_id}: {str(e)}")
            raise
```

Key Features

- Cosine similarity search
- Real-time vector similarity search
- Metadata filtering capabilities
- GDPR-compliant data deletion
- Automatic embedding updates

API Specifications

Core API Endpoints

User Management
- POST /api/v1/users/register
- GET /api/v1/users/{id}/profile
- PUT /api/v1/users/{id}/profile
- DELETE /api/v1/users/{id}/account

Matching System
- POST /api/v1/agent/initialize
- POST /api/v1/agent/find-matches
- GET /api/v1/agent/{id}/status
- PUT /api/v1/agent/{id}/settings

AI Agent Endpoints
- POST /api/v1/agent/conversation
- GET /api/v1/agent/{id}/analysis
- POST /api/v1/agent/{id}/advice
- PUT /api/v1/agent/{id}/settings

Messaging System
- POST /api/v1/messages/send
- GET /api/v1/messages/{id}/status
- POST /api/v1/messages/{id}/analyze
- PUT /api/v1/messages/{id}/edit

API Implementation Example

```
from fastapi import FastAPI, Depends, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List, Optional, Dict, Any

app = FastAPI(title="CYRANO API", version="0.0.1")

security_scheme = HTTPBearer()
supabase = SupabaseClient(os.environ['SUPABASE_URL'],
    os.environ['SUPABASE_SERVICE_KEY'])

class MessageRequest(BaseModel):
    recipient_id: str
    content: str
    conversation_id: Optional[str]
    ai_assistance: bool = True

class MessageAnalysisRequest(BaseModel):
    message_id: str
    analysis_type: str  # 'sentiment', 'compatibility', 'suggestions'
    context: Optional[dict]

class MatchRequest(BaseModel):
    target_user_id: str
    initial_message: Optional[str]

@app.post("/api/v1/agent/find-matches")
@rate_limit("10/minute")
@cache_response(ttl=300)
@log_ai_interaction("matching")
@monitor_performance("/api/v1/agent/find-matches")
@error_handler("/api/v1/agent/find-matches")
@validate_ai_response("/api/v1/agent/find-matches")
@ab_test("matching_algorithm")
@feature_flag("ai_matching")
@analytics("matching_request")
@cache_invalidation("user_{user_id}_matches")
@rate_limit_by_ip("100/hour")
@rate_limit_by_user("50/hour")
@rate_limit_by_endpoint("/api/v1/agent/find-matches")
@rate_limit_by_method("POST")
@rate_limit_by_header("X-Forwarded-For")
@rate_limit_by_cookie("session_id")
@rate_limit_by_query_param("user_id")
@rate_limit_by_path_param("user_id")
@rate_limit_by_body_field("target_user_id")
@rate_limit_by_header("Authorization")
@rate_limit_by_cookie("auth_token")
@rate_limit_by_query_param("target_user_id")
@rate_limit_by_path_param("target_user_id")
@rate_limit_by_body_field("initial_message")
@rate_limit_by_header("User-Agent")
@rate_limit_by_cookie("user_preferences")
@rate_limit_by_query_param("ai_assistance")
@rate_limit_by_path_param("ai_assistance")
@rate_limit_by_body_field("ai_assistance")
@rate_limit_by_header("Accept-Language")
@rate_limit_by_cookie("language_preference")
@rate_limit_by_query_param("language")
@rate_limit_by_path_param("language")
@rate_limit_by_body_field("language")
@rate_limit_by_header("Content-Type")
@rate_limit_by_cookie("content_type_preference")
@rate_limit_by_query_param("content_type")
@rate_limit_by_path_param("content_type")
@rate_limit_by_body_field("content_type")
@rate_limit_by_header("Accept")
@rate_limit_by_cookie("accept_preference")
@rate_limit_by_query_param("accept")
@rate_limit_by_path_param("accept")
@rate_limit_by_body_field("accept")
@rate_limit_by_header("Referer")
@rate_limit_by_cookie("referer_preference")
@rate_limit_by_query_param("referer")
@rate_limit_by_path_param("referer")
@rate_limit_by_body_field("referer")
@rate_limit_by_header("Origin")
@rate_limit_by_cookie("origin_preference")
@rate_limit_by_query_param("origin")
@rate_limit_by_path_param("origin")
@rate_limit_by_body_field("origin")
@rate_limit_by_header("Cookie")
@rate_limit_by_cookie("session_id")
@rate_limit_by_query_param("session_id")
@rate_limit_by_path_param("session_id")
@rate_limit_by_body_field("session_id")
@rate_limit_by_header("Authorization")
@rate_limit_by_cookie("auth_token")
@rate_limit_by_query_param("auth_token")
@rate_limit_by_path_param("auth_token")
@rate_limit_by_body_field("auth_token")
@rate_limit_by_header("User-Agent")
@rate_limit_by_cookie("user_preferences")
@rate_limit_by_query_param("user_preferences")
@rate_limit_by_path_param("user_preferences")
@rate_limit_by_body_field("user_preferences
```